

ECOTRACK TEAM EXECUTION PLAN

Member 3: Super Admin Review and Android Authentication

A detailed backend transaction, authorization, audit, notification, web review, and Android Supabase authentication plan that consumes—but does not modify—Member 2’s database schema.

Primary owner	Team Member 3
Review branch	feature/super-admin-organization-review
Mobile branch	feature/android-authentication
Database dependency	Wait for Member 2 migration handoff
Prepared	10 August 2026
Status	Parallel-start with gated DB integration

Contents

1. Mission and scope
2. Mandatory sources and rules
3. Branches and ownership
4. What can start immediately
5. Database handoff gate
6. Super Admin API contracts
7. Review backend module
8. Approval transaction
9. Decline transaction
10. Audit and notification behavior
11. Authorization and security
12. Super Admin web pages
13. Android project and Supabase client
14. Android magic-link/PKCE flow
15. Android backend API integration
16. Testing strategy
17. Handoff and definition of done

1. Mission and scope

Member 3 owns two separate concerns and therefore uses two separate branches:

1. Super Admin organization review on the backend and web.
2. Supabase authentication in the empty `mobile` Android Studio project.

Core review outcome: an active SUPER_ADMIN can list and inspect pending organization applications, approve or decline exactly once, record reviewer/audit information, notify the requester, and create the requester's first active ORG_ADMIN membership only on approval.

Core mobile outcome: Android uses the same Supabase project and passwordless identity model, securely handles deep-link/PKCE sessions, sends the Supabase access token as a bearer token to the existing Express backend, loads `/auth/me`, persists session state, and signs out correctly.

Excluded

- Database schema and migration ownership.
- Citizen organization application submission.
- GN boundary import.
- Incidents, coordinates, point-to-GN lookup, dashboards and maps.

2. Mandatory sources and rules

- `docs/team-plans/VERY_IMPORTANT_TEAM_COLLABORATION_RULES.txt`
- `docs/IMPORTANT/EcoTrack_Backend_Module_Structure_and_Heavy_Service_Reminder.txt`
- `docs/IMPORTANT/EcoTrack_Complete_System_Flow_Source_of_Truth_v1.txt`
- `database_docs/EcoTrack_Database_v2_Finalization_and_Codex_Instructions.txt`
- `backend/prisma/schema.prisma` after Member 2 handoff
- Current web authentication implementation under `web/src/features/auth`
- Current backend authentication routes/middleware/services/repositories

Do not: change `schema.prisma`, create migrations, edit Member 2's migration, create approval seeds as a substitute for runtime logic, trust a frontend role, put a Supabase secret/service-role key in Android, or implement review without CASL permission from Member 1.

3. Branches and ownership

Review branch

```
git fetch origin
git switch integration/organization-onboarding
git pull --ff-only origin integration/organization-onboarding
git switch -c feature/super-admin-organization-review
```

Android branch

```
git fetch origin
git switch integration/organization-onboarding
git pull --ff-only origin integration/organization-onboarding
git switch -c feature/android-authentication
```

Do not combine Android build files and backend review changes in one branch or pull request. They have different dependencies, test commands and merge risks.

Owned paths

```
backend/src/modules/organizations/review/**
backend/src/modules/notifications/** (if assigned)
web/src/features/organizations/review/**
mobile/**
```

Shared/not-owned paths

```
backend/prisma/schema.prisma          -> Member 2
backend/prisma/migrations/**          -> Member 2
backend/src/authorization/**           -> Member 1
backend/src/app.ts                     -> Member 1 integration
backend/package.json                   -> Member 1 integration
web/src/App.tsx                        -> Member 1 integration
web/src/features/organizations/application/** -> Member 2
```

4. What can start immediately

Member 3 does not need to remain idle while Member 2 creates the schema. These tasks can proceed with agreed interfaces and fake data:

- Create API request/response types for pending list, details, approve and decline.
- Create route/controller/service interfaces with fake repository dependencies.
- Write service tests against fake repositories.
- Create Super Admin list/detail web pages using typed mock data.

- Create confirmation dialogs and loading/success/error states.
- Create the Android Studio project under `mobile`.
- Implement Android Supabase client, login UI, deep-link handling, session state, backend profile request and sign-out.

Wait for Member 2 only before writing the real Prisma review repository and database transaction.

5. Database handoff gate

Do not begin real review repository code until Member 2 confirms:

- ☐ New schema and migration are committed.
- ☐ Migration applies successfully and status is current.
- ☐ Prisma client is generated.
- ☐ Organization application creates a PENDING_REVIEW Organization.
- ☐ Selected GN links exist as pending OrganizationServiceArea records.
- ☐ AdministrativeArea and service-area model/enums are documented.
- ☐ Notification model required by review exists.
- ☐ A safe test/application setup procedure exists.

After pulling the merged schema:

```
cd backend
npx prisma validate
npx prisma generate
npx prisma migrate status
npm run typecheck
npm run db:check
```

These commands consume/check Member 2's migration; they do not authorize Member 3 to create another migration.

6. Super Admin API contracts

Endpoint	Purpose	Permission
GET /api/v1/super-admin/organization-applications	Paginated/filterable pending list	Review OrganizationApplication
GET /api/v1/super-admin/organization-applications/:organizationId	Review-safe details, documents and selected GN areas	Review OrganizationApplication
POST /api/v1/super-admin/organization-applications/:organizationId/approve	Approve in one transaction	Approve OrganizationApplication
POST /api/v1/super-admin/organization-applications/:organizationId/decline	Decline with notes in one transaction	Decline OrganizationApplication

Endpoint	Purpose	Permission
GET /api/v1/notifications	Requester reads own notifications	ReadOwn Notification
PATCH /api/v1/notifications/:notificationId/read	Requester marks own notification read (optional milestone)	Update own Notification

Safe response fields

Return organization application data, requester display/contact data required for review, GN area names/codes, document metadata required for review, status/review notes and timestamps. Never return Supabase tokens, auth secrets, storage service credentials, internal stack traces, or unrelated profiles.

7. Review backend module

```

backend/src/modules/organizations/review/
  review.types.ts
  review.validation.ts
  review.dependencies.ts
  review.routes.ts
  controllers/
    listPendingOrganizationApplications.controller.ts
    getOrganizationApplicationForReview.controller.ts
    approveOrganizationApplication.controller.ts
    declineOrganizationApplication.controller.ts
  services/
    listPendingOrganizationApplications.service.ts
    getOrganizationApplicationForReview.service.ts
    approveOrganizationApplication.service.ts
    declineOrganizationApplication.service.ts
  repositories/
    organizationReview.repository.ts
  review.integration.test.ts

```

Layer responsibilities

Layer	Responsibility
Route	Method/path plus authenticate, ability and authorize middleware order.
Controller	Parse route/body, call service, return 200 and safe DTO.
Validation	UUID, review notes and filter/pagination validation through Zod.
Service	State-transition and business invariants; maps domain conflicts to ApplicationError.
Repository	Prisma reads and atomic approval/decline transactions.

8. Approval transaction

Approval must be atomic and conditional. The application must still be `PENDING_REVIEW` when the transaction writes. This prevents two Super Admin sessions from approving/declining the same application concurrently.

```
authenticate active user
-> CASL authorizes Approve
-> validate organization ID and optional notes
-> Prisma transaction:
  conditionally transition Organization PENDING_REVIEW -> ACTIVE
  set reviewedById from authenticated Super Admin profile
  set reviewedAt and activatedAt
  update pending selected service areas -> ACTIVE/approved
  create requester OrganizationMembership:
    role = ORG_ADMIN
    status = ACTIVE
    source = FIRST_ADMIN
  create approval AuditLog
  create ORGANIZATION_APPROVED Notification for requester
-> commit
-> return safe approved result
```

Critical checks

- Reviewer ID comes from `request.authentication.profile.id`, never request body.
- Organization must exist and still be pending.
- Requester profile must still exist and be eligible for membership.
- Selected service areas must be in an approvable state.
- Unique organization/user membership constraint must be respected.
- If the requester is already a member because of data inconsistency, fail safely and investigate; do not create duplicate membership.
- Transaction rollback must leave no partially active organization, missing admin, audit or notification.

Concurrency strategy

Use a conditional update/updateMany or another reviewed transaction pattern that proves the current state is pending. If zero rows transition, return a conflict:

```
409 ORGANIZATION_REVIEW_CONFLICT
"This organization application has already been reviewed or changed."
```

9. Decline transaction

```
authenticate active user
-> CASL authorizes Decline
-> validate organization ID and review notes
-> Prisma transaction:
    conditionally transition Organization PENDING_REVIEW -> DECLINED
    set reviewer, reviewedAt and reviewNotes
    transition pending service areas -> DECLINED
    create decline AuditLog
    create ORGANIZATION_DECLINED Notification
    DO NOT create membership
    DO NOT set activatedAt
-> commit
-> return safe declined result
```

Decide with the team whether review notes are mandatory for decline; recommended: require a useful bounded note. Do not include sensitive internal-only comments in the user-visible notification unless explicitly separated.

10. Audit and notification behavior

Audit log

Field	Approval/decline value
actorUserId	Authenticated Super Admin profile ID
organizationId	Reviewed organization ID
action	ORGANIZATION_APPROVED or ORGANIZATION_DECLINED
entityType	Organization
entityId	Reviewed organization ID
metadata	Safe old/new status, selected-area count, reason code; no secrets

Notification

- Recipient is `organization.requestedByUserId` , not body data.
- Approval message states organization became active and requester is now ORG_ADMIN.
- Decline message states declined status and safe review notes/reason.
- Notification is created inside the same transaction as review state.
- Notification listing filters by authenticated recipient ID.
- WebSockets are not required; load/poll notifications through REST for this milestone.

11. Authorization and security

Member 1 defines action and subject constants. Member 3 consumes them; do not create a second local permission vocabulary.

Review route order

```
router.post(
  "/super-admin/organization-applications/:organizationId/approve",
  authenticate,
  buildAbility,
  authorize(Actions.Approve, Subjects.OrganizationApplication),
  approveOrganizationApplicationController,
);
```

Security requirements

- Active SUPER_ADMIN platform role is required for all review routes.
- Normal USER and ORG_ADMIN membership alone are insufficient.
- Frontend button visibility is not authorization.
- Review list/detail must not leak to unauthorized direct requests.
- Reviewer and actor IDs always come from authentication context.
- All errors use ApplicationError and central error middleware.
- Unexpected Prisma errors do not expose database details.
- Use pagination and bounded filters for lists.

12. Super Admin web pages

Use the existing `ecotrack-srs-mockup` Super Admin verification screen as a visual reference, then connect it to real bearer-authenticated APIs.

Required UI

- Pending-application list with organization name, requester, submitted date and selected-area count.
- Review details with official contact data, description, registration information, document metadata and GN areas.
- Approve confirmation.
- Decline confirmation with validated review-notes field.
- Loading, empty, error, conflict and success states.
- Status update/removal from pending list after success.

Frontend rules

- Use existing `apiClient.ts` and Supabase session hook.
- Do not send reviewer ID, platform role, membership role, new status or timestamps from the browser.

- Disable actions while request is pending to reduce double submission.
- Still handle backend 409 because UI disabling cannot prevent concurrent reviewers.
- Export pages/components for Member 1; do not replace production `App.tsx`.
- Use accessible dialogs, labels, focus handling and keyboard operation.

13. Android project and Supabase client

The repository's `mobile` directory is currently empty. Create the Android Studio project there on the separate mobile branch. Use Kotlin and the agreed UI approach (Jetpack Compose if selected by the team).

Initial requirements

- Internet permission in `AndroidManifest.xml`.
- Supabase Kotlin client and Auth module.
- Compatible Ktor client engine and Kotlin serialization as required by the chosen library version.
- Supabase project URL and publishable key supplied through a non-secret build configuration.
- No Supabase secret/service-role key in mobile source, resources or build output.
- Backend API base URL configurable for emulator/device development.

Networking note: Android emulator cannot use `localhost` to reach the developer computer. Use the emulator host address (commonly `10.0.2.2`) or an agreed LAN/development URL, and expose the backend carefully only as required.

Supabase client responsibilities

- Install Auth.
- Use PKCE flow.
- Configure deep-link scheme and host.
- Persist/restore session through the library.
- Observe authenticated, initializing, refresh failure and signed-out states.

14. Android magic-link and PKCE flow

Supabase dashboard

Add the agreed Android deep-link URL to Supabase Authentication redirect URLs. Use an app-specific scheme/host, for example:

```
ecotrack://auth-callback
```

The exact scheme and host must match Android manifest intent filters and the Supabase Auth client configuration.

Android flow

```
User enters email
-> Android calls Supabase signInWith(OTP) for email
-> Custom SMTP sends magic link
-> user opens link on Android
-> Android deep link opens EcoTrack
-> MainActivity passes intent to Supabase handleDeeplinks(intent)
-> PKCE code/session is handled by the Auth plugin
-> sessionStatus becomes Authenticated
-> Android sends access token to Express /api/v1/auth/me
-> backend verifies token and returns EcoTrack profile
-> authenticated mobile UI appears
```

Conceptual Kotlin configuration

```
val supabase = createSupabaseClient(
    supabaseUrl = BuildConfig.SUPABASE_URL,
    supabaseKey = BuildConfig.SUPABASE_PUBLISHABLE_KEY,
) {
    install(Auth) {
        flowType = FlowType.PKCE
        scheme = "ecotrack"
        host = "auth-callback"
    }
}
```

Use the current Supabase Kotlin documentation while implementing because Gradle coordinates and APIs must match the selected library version.

Deep-link handling

- Add the manifest intent filter for the exact scheme/host.
- Call `supabase.handleDeeplinks(intent)` early in activity startup as required by the library.
- Handle a new intent if the activity is reused.
- Display expired/invalid link errors safely.
- PKCE requires the code verifier from the initiating client; test the complete same-device flow.
- Do not log magic-link parameters, access tokens or refresh tokens.

15. Android backend API integration

Request

```
GET /api/v1/auth/me
Authorization: Bearer <Supabase access token>
Accept: application/json
```

The mobile application must use the same backend contract as web. It does not query private EcoTrack application tables directly through Supabase. The backend owns profile provisioning, account-status checks and authorization.

Mobile auth state

State	UI/behavior
Initializing	Loading state; do not briefly show protected UI.
Not authenticated	Email magic-link form.
Email sent	Clear same-device link instructions and resend cooldown.
Authenticated, profile loading	Send bearer request to /auth/me.
Authenticated and active	Show authenticated profile/home appropriate to milestone.
Account inactive	Show backend message; do not show protected app.
Refresh failure	Clear/repair session and return to sign-in safely.
Signed out	Remove session state and show login.

Sign-out

Call Supabase sign-out, observe the session state, clear in-memory EcoTrack profile data, and navigate to login. Test for stale-request races similar to the web fix: an older profile request must not overwrite the newer signed-out state.

16. Testing strategy

Review service/repository tests

- Normal USER receives 403 for every review endpoint.
- Active SUPER_ADMIN can list pending and read review details.
- Suspended Super Admin is rejected.
- Approval transitions exactly one pending organization.
- Approval activates selected service areas.
- Approval creates exactly one FIRST_ADMIN membership.
- Approval creates audit and requester notification.
- Decline creates no membership and no activatedAt.
- Decline creates audit and requester notification.
- Second/concurrent review returns 409 and creates no duplicates.
- Any transaction failure rolls back every write.
- Reviewer ID/body spoofing is impossible.
- Notification list returns only authenticated recipient records.

Web tests/manual checks

- Pending list loading/empty/error/success states.
- Detail displays selected GN names/codes and review metadata.
- Approve/decline buttons disabled while pending.
- 409 conflict is explained and list refreshes.
- Normal user cannot see review navigation and direct API still returns 403.
- Build and lint pass.

Android tests/manual checks

- App contains only publishable key, never service key.
- Email sends through configured SMTP.
- Magic link opens the app through exact deep-link scheme/host.
- PKCE session completes on same device.
- Session survives activity/app restart as intended.
- Bearer token reaches backend and /auth/me returns profile.
- Invalid/expired token produces safe login/error state.
- Sign-out remains signed out after restart.
- Access/refresh tokens never appear in logs.

Backend final commands

```
cd backend
npm run typecheck
npm test
npx tsx --test src/modules/organizations/review/review.integration.test.ts
npm run build
npx prisma validate
npx prisma migrate status
npm run db:check

cd ../web
npm run build
npm run lint
```

17. Handoff and definition of done

Review handoff to Member 1

- ☐ Router factory exported without changing production app.ts.
- ☐ Dependencies exported/documented.
- ☐ API DTOs documented.
- ☐ Approval/decline transaction and conflict behavior documented.
- ☐ Authorization constants imported from Member 1.

- ☐ Web pages exported without replacing production App.tsx.
- ☐ Tests and commands included in pull request.

Review definition of done

- ☐ Member 2 schema used without schema/migration edits.
- ☐ Only active SUPER_ADMIN may review.
- ☐ Approval is atomic, conditional and creates first admin/audit/notification.
- ☐ Decline is atomic, conditional, creates no membership and records audit/notification.
- ☐ Concurrent/double review produces 409 with no duplicate side effects.
- ☐ Super Admin pages use real API and EcoTrack visual style.
- ☐ Backend and web checks pass.

Android definition of done

- ☐ Android project committed only on mobile branch.
- ☐ Publishable key and project URL configured without secrets.
- ☐ PKCE and deep-link handling implemented.
- ☐ Magic link, session restore, /auth/me, inactive-account handling and sign-out tested.
- ☐ No incident/map functionality added.
- ☐ Build/setup instructions documented for another team member.

Reference material

- Local source: `docs/IMPORTANT/EcoTrack_Backend_Module_Structure_and_Heavy_Service_Reminder.txt`
- Local source: `database_docs/EcoTrack_Database_v2_Finalization_and_Codex_Instructions.txt`
- Supabase Kotlin initialization and deep links: <https://supabase.com/docs/reference/kotlin/initializing>
- Supabase Kotlin email OTP/magic link: <https://supabase.com/docs/reference/kotlin/auth-signinwithotp>
- Supabase Kotlin auth session events: <https://supabase.com/docs/reference/kotlin/auth-onauthstatechange>
- Supabase Android/Kotlin quickstart: <https://supabase.com/docs/guides/getting-started/quickstarts/kotlin>

EcoTrack team plan - Member 3. Review and Android work remain separate branches. Schema changes are requested from Member 2, authorization contracts come from Member 1, and no incident/map work is part of this milestone.